



**RV
UNIVERSITY**

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

**New-Age Global
University for
Liberal Education**

**School of Computer Science and
Engineering**

Where Ideas Ignite Minds

**Laboratory Record Book
OPERATING SYSTEMS -[CS1103]**

**Bachelor of Technology (B.Tech.) (Hons.)
Academic Year 2025-26**

RV UNIVERSITY

RASHTREEYA SIKSHANA SAMITHI TRUST

SCHOOL OF COMPUTER SCIENCE AND

ENGINEERING RV UNIVERSITY

BENGALURU – 560059



COURSE NAME: OPERATING SYSTEM

PROGRAM: B.TECH (HONS.)

COURSE CODE: CS1103

SEMESTER:II

LABORATORY RECORD

BOOK 2025-2026

RV UNIVERSITY
School of Computer Science and Engineering
Bengaluru – 560 059



an initiative of RV EDUCATIONAL INSTITUTIONS

LABORATORY CERTIFICATE

This is to certify that Mr./Ms. _____

USN: _____ Semester: _____ has satisfactorily
completed the course of experiments in Practical Operating System (CS1103) prescribed by the School
of Computer Science and Engineering during the year 2025-26.

Marks	
Maximum	Obtained
25	

Signature of Faculty in-charge

Program Director

Date:

Vision

The School of Computer Science and Engineering at RV University envisions the Operating Systems course as a cornerstone of computing education, equipping students with a deep understanding of system architecture, resource management, and process coordination. This course fosters a strong foundation in theoretical principles and practical applications, enabling students to design, analyze, and optimize modern operating systems. With exposure to hands-on projects, research-led education, and industry linkages, the aspiration is to develop problem solvers and innovators who can create computing solutions that are efficient, secure, and scalable. Technological trends like virtualization, cloud computing, and cybersecurity are imparted in a way that prepares students to flourish in academia, industry, and entrepreneurship, thereby powering the advancement of technology and making a difference in society.

Mission

- * **Fundamental Understanding-** An OS itself is a rich set of concepts involving process management, memory management, file systems, and security under which students would have a considerable theoretical foundation.
- * **Hands-On Learning-** Students will have hands-on experiences with lab experiments, coding assignments, and project work involving actual systems.
- * **Industry Relevance-** Teaching concerning current industry practices offers students some understanding of topics in virtualization, cloud computing, and cybersecurity.
- * **Research & Innovation-** A research perspective fosters a contribution towards solving system-level problems and allows an exploration of potential emerging areas in OS development.
- * **Performance Optimization-** Students learn how to analyze and optimize the OS component for efficiency, scalability, and reliability.
- * **Interdisciplinary Approach-** Exploring the OS interplay with computer architecture, embedded systems, and networking helps students develop an understanding that is free from hardware-software divides.
- * **Security & Reliability-** Systems must be designed to be secure and to tolerate failures to protect against data corruption and system integrity.
- * **Collaboration & Teamwork-** Collaboration fosters innovation and teamwork skills in students through group projects, open-source contributions, and hackathons.

- * Ethical Computing- Awareness of the ethical considerations of system design including privacy, fair allocation of resources, and sustainable computing must be developed in students.
- * Career Readiness- A broad range of skills, knowledge, and experiences are provided through internships, collaboration with industry, and skill-build workshops to prepare students for career opportunities in the fields of software engineering, system administration, cybersecurity, and research.

Nomenclature for Program Educational Objectives (PEOs):

PEO1: Graduates will be able to demonstrate excellence in the field of Computer Science and Engineering through advanced research and entrepreneurship.

PEO2: Graduates will be able to achieve excellence in innovation through national and international academic and industry collaborations.

PEO3: Graduates will be able to actively engage in cutting-edge interdisciplinary and multidisciplinary research.

PEO4: Graduates will be able to function effectively as a talented pool of ethical, sustainable, self-driven, and life-long empathetic problem solvers.

Nomenclature for Program Outcomes (POs):

Engineering Graduates will be able to:

PO1: Engineering Knowledge. Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem Analysis. Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/Development of Solutions. Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.

PO4: Conduct Investigations of Complex Problems. Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern Tool Usage. Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The Engineer and Society. Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues, and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and Sustainability. Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Individual and Teamwork. Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO9: Ethics. Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO10: Communication. Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project Management and Finance. Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-Long Learning. Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Nomenclature for Course Outcomes (COs):

Course Outcomes: After completing the course, the students will be able to:

CO1: Understand the structural components and core functionalities of an operating system.

CO2: Apply the process management techniques and scheduling algorithms to ensure efficient execution and resource allocation.

CO3: Apply synchronization techniques and design solutions to prevent or resolve deadlocks in concurrent systems.

CO4: Illustrate the memory management strategies, principles of file system design, and security mechanisms for efficient performance, resource utilization, and reliable data handling in operating systems.

Course Outcomes to Program Outcome Mapping:

Course Outcomes		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
1.	Understand the structural components and core functionalities of an operating system.	3	2	2		2							2
2.	Apply process management techniques and scheduling algorithms to ensure efficient execution and resource allocation.	3	3	2		3				2			2
3.	Apply synchronization techniques and design solutions to prevent or resolve deadlocks in concurrent systems.	3	3	2		2				2			2
4.	Illustrate the memory management strategies, principles of file system design, and security mechanisms for efficient performance, resource utilization, and reliable data handling in operating systems.	3	3	2		2				2	2		2

Note: High: 3, Medium: 2, Low: 1

Lab Marks Rubrics:

Table-1

Practical Execution rubrics (Max: 6 marks)					
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor
	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)
	Execution (2 Marks)	Observations	The student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The student has not executed the program. (0 M)
	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)
Quiz Rubrics (Max: 4 marks)					
Sl. No	Criteria	Measuring Methods	Excellent (2 Marks)	Good (1 Mark)	Poor (0.5 Marks)
	Conceptual Understanding of Operating System (2 Marks)	Quiz	Thorough explanation of concepts	Adequate explanation with some gaps	Unable to explain tools/concepts clearly

	Application of Problem-Solving Techniques (2 Marks)	Quiz	Describes and justifies the selection of specific services	Identifies applicable methods with minor gaps in justification	Lacks clarity or incorrect application
--	--	-------------	--	--	--

Table-2

Sl. No.	Criteria	Measuring Methods	Excellent	Good	Satisfactory	Poor
1	Conceptual Understanding (6 Marks)	Quiz	Answers show clear and accurate understanding of all key concepts (6 M)	Minor conceptual errors (5 M)	Basic understanding with noticeable gaps (3 M)	Poor or incorrect understanding (1 M)
2	Application of Concepts (4 Marks)	Quiz	Correctly applies concepts to problem-based questions (4 M)	Mostly correct application (3 M)	Limited or partially correct application (2 M)	Unable to apply concepts (1 M)
3	Analytical & Logical Reasoning (3 Marks)	Quiz	Demonstrates strong reasoning and justification (3 M)	Reasoning mostly logical (2 M)	Limited analytical ability (1 M)	Illogical or random answers (0 M)
4	Accuracy & Completeness (2 Marks)	Quiz	All answers accurate and complete (2 M)	Few minor mistakes (1.5 M)	Several incorrect/incomplete answers (1 M)	Mostly incorrect (0 M)

List of Laboratory Programs:

Sl. no	Programs
1.	1.1) Exploring basic Linux commands and Shell programs. 1.2) Execute the C program using Command Line Arguments. i) Add numbers ii) Find the average of the numbers iii) Find the minimum from the given numbers 1.3) Write a program in C to print the environment variable.
2.	Execute C programs to implement Process system calls using the fork system call, exec, wait, create process, and terminate the process.
3.	Execute C programs to simulate CPU scheduling algorithms to find turnaround and waiting time. 3.1) First Come First Served (FCFS) algorithm 3.2) Shortest Job First (SJF) algorithm 3.3) Round Robin (RR) algorithm 3.4) Priority Scheduling algorithm
4.	Execute C programs for Thread Creation and Management using the POSIX pthread library.
5.	Execute a C Program for IPC using the pipe () function.
6.	Execute a C program to simulate the producer-consumer problem using semaphores.
7.	Execute the Banker's Algorithm for deadlock avoidance using the C program.
8.	Execute C programs to simulate Page Replacement Algorithms. 9.1) First In First Out (FIFO) algorithm 9.2) Least Recently Used (LRU) algorithm
9.	Execute C programs to display disk scheduling algorithms. 11.1) First Come First Serve (FCFS) algorithm 11.2) Shortest Seek Time First (SSFT) algorithm 11.3) SCAN algorithm

Part-A: Index

Sl.No	Program Name	Date	Record Mark (6M)	Quiz (4M)	Total Marks(10)	Sign
1						
2						
3						
4						
5						
6						
7						
8						
9						
	Total (10)					

Part-B (Assignment and Presentation)

Sl.No	QUIZ		Mock Assignment
1	Lab(1-3)		
2	Lab(4-6)		
3	Lab(7-9)		
Total			

Laboratory Programs:

1.1) Exploring basic Linux commands and Shell programs. (Include datasheets for pseudocode/code with output, and the output screenshots also).

Solution: (paste your data sheets here)

1.2) Execute the C program using Command Line Arguments.

- i. Add numbers**
- ii. Find the minimum from the given numbers**
- iii. Find the average of the given number**

(Include datasheets for pseudocode/ code with output, and the output screenshots also).

Solution: (paste your data sheets here)

**1.3) Write a program in C to print the environment variable.
(Include datasheets for pseudocode/ code with output, and the output screenshots also).**

Solution: (paste your data sheets here)

Lab Assessment (10 Marks)							
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor	Marks	Remarks
1.	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)		
2.	Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The Student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The Student has not executed the program. (0 M)		
3.	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)		
4.	Signature with Date					Total Marks	

2. **Execute C programs to implement Process system calls using the fork system call, exec, wait, create process, and terminate the process. (Include datasheets for pseudocode/ code with output, and the output screenshots also).**

Solution: (paste your data sheets here)

Lab Assessment (10 Marks)							
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor	Marks	Remarks
1.	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)		
2.	Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The Student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The Student has not executed the program. (0 M)		
3.	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)		
4.	Signature with Date					Total Marks	

3. **Execute C programs to simulate CPU scheduling algorithms to find turnaround and waiting time.**
 - 3.1. **First Come First Served (FCFS) algorithm**
 - 3.2. **Shortest Job First (SJF) algorithm**
 - 3.3. **Round Robin (RR) algorithm**
 - 3.4. **Priority Scheduling algorithm**

(Include datasheets for pseudocode/ code with output, and the output screenshots also).

Solution: (paste your data sheets here)

Lab Assessment (10 Marks)							
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor	Marks	Remarks
1.	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)		
2.	Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The Student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The Student has not executed the program. (0 M)		
3.	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)		
4.	Quiz (4 Marks)						
5	Signature with Date					Total Marks	

4. **Execute C programs for Thread Creation and Management using the POSIX pthread library. (Include datasheets for pseudocode/ code with output, and the output screenshots also).**

Solution: (paste your data sheets here)

Lab Assessment (10 Marks)							
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor	Marks	Remarks
1.	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)		
2.	Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The Student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The Student has not executed the program. (0 M)		
3.	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)		
4.	Signature with Date					Total Marks	

5. **Execute a C Program for IPC using the pipe () function. (Include datasheets for pseudocode/ code with output, and the output screenshots also).**

Solution: (paste your data sheets here)

Lab Assessment (10 Marks)							
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor	Marks	Remarks
1.	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)		
2.	Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The Student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The Student has not executed the program. (0 M)		
3.	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)		
4.	Signature with Date					Total Marks	

6. **Execute a C program to simulate the producer-consumer problem using semaphores. (Include datasheets for pseudocode/ code with output, and the output screenshots also).**

Solution: (paste your data sheets here)

Lab Assessment (10 Marks)							
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor	Marks	Remarks
1.	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)		
2.	Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The Student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The Student has not executed the program. (0 M)		
3.	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)		
4.	Quiz (4 Marks)						
5.	Signature with Date					Total Marks	

7. Execute the Banker's Algorithm for deadlock avoidance using the C program. (Include datasheets for pseudocode/ code with output, and the output screenshots also).

Solution: (paste your data sheets here)

Lab Assessment (10 Marks)							
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor	Marks	Remarks
1.	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)		
2.	Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The Student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The Student has not executed the program. (0 M)		
3.	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)		
4.	Signature with Date					Total Marks	

8. Execute C programs to simulate Page Replacement Algorithms:

8.1. First In First Out (FIFO) algorithm

8.2. Least Recently Used (LRU) algorithm

(Include datasheets for pseudocode/ code with output, and the output screenshots also).

Solution: (paste your data sheets here)

Lab Assessment (10 Marks)							
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor	Marks	Remarks
1.	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)		
2.	Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The Student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The Student has not executed the program. (0 M)		
3.	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)		
4.	Signature with Date					Total Marks	

9. Execute C programs to display disk scheduling algorithms.

- 9.1. First Come First Serve (FCFS) algorithm**
- 9.2. Shortest Seek Time First (SSTF) algorithm**
- 9.3. SCAN algorithm**

(Include datasheets for pseudocode/ code with output, and the output screenshots also).

Solution: (paste your data sheets here)

Lab Assessment (10 Marks)							
Sl. No	Criteria	Measuring Methods	Excellent	Good	Poor	Marks	Remarks
1.	Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. (2 M)	The student has a sufficient understanding of the requirements for the problem. (1 M)	The student does not have a clear understanding of the requirements for the problem. (0 M)		
2.	Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. (2 M)	The Student demonstrates the execution of the program without optimization of the code and shows the output. (1 M)	The Student has not executed the program. (0 M)		
3.	Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. (2 M)	Documentation with few comments and few output cases are noted in data sheets. (1 M)	Documentation with no comments and no output cases are noted in data sheets. (0 M)		
4.	Quiz (4 Marks)						
5	Signature with Date					Total Marks	

RV University Vision and Mission

Vision

To be a world-class, tech-driven, global university for liberal education, empowering citizens of tomorrow.

Mission

- **Strive for excellence in teaching, research, capacity building, community engagement and to build a reputation in the academic world as a thought leader in various disciplines; benchmark against top universities of the world.**
- **Use digital and other technologies relevant to teaching-learning process and to do advanced research.**
- **Creation of a vibrant and diverse community with faculty members and students hailing from different parts of the country and the world.**
- **Forge a large number of national and international collaborations to create learning opportunities and mobility for the students and faculty members.**
- **Make higher education accessible to all. Offer a multi-disciplinary learning environment in a diversity of knowledge streams and skills catering to the existing and emerging needs from a learner's context. To create a research rewarding environment that seeks and encourages inter-disciplinary expertise and institution building enterprise among faculty members, while consistently forging cooperation between global and Indian institutions of eminence.**
- **Expose students to the best of theory and practice.**
- **Evolve a diverse talent pool of informed, self-driven, empathetic & inclusive problem solvers.**



**RV
UNIVERSITY**

**RV Vidyanikethan Post, 8th Mile, Mysuru Road, Bengaluru -
560 059 For general enquiry: enquiry@rvu.edu.in | Phone:
+91 95136 73778**

RV University

RVUniversity

rv-university

RV.University1

rv.university



Operating Systems Lab (CS1103) - Program Codes & Execution

1.1) Exploring basic Linux commands and Shell programs.

```
#!/bin/bash
echo "Exploring basic Linux commands..."
echo "Current working directory:"
pwd
echo "Listing files:"
ls -l
echo "Shell program executed successfully."
```

Output - 1.1

```
Exploring basic Linux commands...  
Current working directory:  
/home/student/oslab  
Listing files:  
total 4  
-rwxr-xr-x 1 root root 145 May 2 08:21 script.sh  
Shell program executed successfully.
```

1.2) Execute the C program using Command Line Arguments.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Please provide numbers as arguments.\n");
        return 1;
    }
    int sum = 0, min = atoi(argv[1]);
    for (int i = 1; i < argc; i++) {
        int val = atoi(argv[i]);
        sum += val;
        if (val < min) min = val;
    }
    printf("Sum: %d\n", sum);
    printf("Average: %.2f\n", (float)sum / (argc - 1));
    printf("Minimum: %d\n", min);
    return 0;
}
```

Output - 1.2

```
Sum: 45  
Average: 11.25  
Minimum: 5
```

1.3) Write a program in C to print the environment variable.

```
#include <stdio.h>

extern char **environ;

int main() {
    printf("Environment Variables:\n");
    for (int i = 0; environ[i] != NULL && i < 4; i++) {
        printf("%s\n", environ[i]);
    }
    printf("... (truncated for brevity)\n");
    return 0;
}
```

Output - 1.3

```
Environment Variables:  
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin  
USER=student  
HOME=/home/student  
SHELL=/bin/bash  
... (truncated for brevity)
```


2) Execute C programs to implement Process system calls using the fork system call, exec, wait...

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();
    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    } else if (pid == 0) {
        printf("Child Process (PID: %d): Executing 'ls'\n", getpid());
        execlp("ls", "ls", "-l", NULL);
        exit(0);
    } else {
        wait(NULL);
        printf("Parent Process (PID: %d): Child terminated.\n", getpid());
    }
    return 0;
}
```

Output - 2

```
Child Process (PID: 1244): Executing 'ls'
total 8
-rwxr-xr-x 1 root root 8432 May 2 08:21 process.exe
-rw-r--r-- 1 root root  430 May 2 08:21 process.c
Parent Process (PID: 1243): Child terminated.
```

3.1) First Come First Served (FCFS) algorithm

```
#include <stdio.h>

int main() {
    int bt[] = {24, 3, 3}, n = 3;
    int wt[3] = {0}, tat[3];
    float awt = 0, atat = 0;

    for (int i = 1; i < n; i++)
        wt[i] = bt[i-1] + wt[i-1];

    printf("P\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    }
    printf("Avg WT: %.2f, Avg TAT: %.2f\n", awt/n, atat/n);
    return 0;
}
```

Output - 3.1

P	BT	WT	TAT
P1	24	0	24
P2	3	24	27
P3	3	27	30

Avg WT: 17.00, Avg TAT: 27.00

3.2) Shortest Job First (SJF) algorithm

```
#include <stdio.h>

int main() {
    int bt[] = {6, 8, 7, 3}, p[] = {1, 2, 3, 4}, n = 4;
    int wt[4] = {0}, tat[4], temp;
    float awt = 0, atat = 0;

    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }
        }
    }

    for (int i = 1; i < n; i++) wt[i] = bt[i-1] + wt[i-1];

    printf("P\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        awt += wt[i]; atat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
    }
    printf("Avg WT: %.2f, Avg TAT: %.2f\n", awt/n, atat/n);
    return 0;
}
```

Output - 3.2

P	BT	WT	TAT
P4	3	0	3
P1	6	3	9
P3	7	9	16
P2	8	16	24

Avg WT: 7.00, Avg TAT: 13.00

3.3) Round Robin (RR) algorithm

```
#include <stdio.h>

int main() {
    int bt[] = {10, 5, 8}, rem_bt[] = {10, 5, 8}, n = 3, qt = 2;
    int wt[3] = {0}, tat[3], t = 0, done = 0;
    float awt = 0, atat = 0;

    while (done < n) {
        for (int i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                if (rem_bt[i] > qt) {
                    t += qt; rem_bt[i] -= qt;
                } else {
                    t += rem_bt[i]; wt[i] = t - bt[i];
                    rem_bt[i] = 0; done++;
                }
            }
        }
    }

    printf("P\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        awt += wt[i]; atat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    }

    printf("Avg WT: %.2f, Avg TAT: %.2f\n", awt/n, atat/n);
    return 0;
}
```

Output - 3.3

P	BT	WT	TAT
P1	10	13	23
P2	5	10	15
P3	8	13	21

Avg WT: 12.00, Avg TAT: 19.67

3.4) Priority Scheduling algorithm

```
#include <stdio.h>

int main() {
    int bt[] = {10, 1, 2, 1, 5}, pr[] = {3, 1, 4, 5, 2}, p[] = {1, 2, 3, 4, 5}, n = 5;
    int wt[5] = {0}, tat[5], temp;
    float awt = 0, atat = 0;

    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n; j++) {
            if (pr[i] > pr[j]) {
                temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }
        }
    }

    for (int i = 1; i < n; i++) wt[i] = bt[i-1] + wt[i-1];

    printf("P\tBT\tPR\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        awt += wt[i]; atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\n", p[i], bt[i], pr[i], wt[i], tat[i]);
    }

    printf("Avg WT: %.2f, Avg TAT: %.2f\n", awt/n, atat/n);
    return 0;
}
```

Output - 3.4

P	BT	PR	WT	TAT
P2	1	1	0	1
P5	5	2	1	6
P1	10	3	6	16
P3	2	4	16	18
P4	1	5	18	19

Avg WT: 8.20, Avg TAT: 12.00

4) Execute C programs for Thread Creation and Management using the POSIX pthread library.

```
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the created thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;
    printf("Main thread creating a new thread...\n");
    pthread_create(&thread, NULL, thread_function, NULL);
    pthread_join(thread, NULL);
    printf("Thread management complete.\n");
    return 0;
}
```

Output - 4

```
Main thread creating a new thread...  
Hello from the created thread!  
Thread management complete.
```

5) Execute a C Program for IPC using the pipe () function.

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    char buffer[50];
    pipe(fd);

    if (fork() == 0) {
        close(fd[0]);
        char msg[] = "Data from child via pipe";
        write(fd[1], msg, strlen(msg) + 1);
        printf("Child: Wrote data to pipe.\n");
    } else {
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        printf("Parent: Read -> '%s'\n", buffer);
    }
    return 0;
}
```

Output - 5

```
Child: Wrote data to pipe.  
Parent: Read -> 'Data from child via pipe'
```

6) Execute a C program to simulate the producer-consumer problem using semaphores.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define MAX 5
int buffer[MAX], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    for(int i = 1; i <= 3; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        printf("Producer produced: %d\n", i);
        in = (in + 1) % MAX;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void* consumer(void* arg) {
    for(int i = 1; i <= 3; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % MAX;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, MAX);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
```

```
    pthread_join(prod, NULL);  
    pthread_join(cons, NULL);  
    return 0;  
}
```


Output - 6

```
Producer produced: 1
Producer produced: 2
Consumer consumed: 1
Producer produced: 3
Consumer consumed: 2
Consumer consumed: 3
```

7) Execute the Banker's Algorithm for deadlock avoidance using the C program.

```
#include <stdio.h>

int main() {
    int n = 5, m = 3;
    int alloc[5][3] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};
    int max[5][3] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
    int avail[3] = {3, 3, 2}, need[5][3], safe[5];
    int finish[5] = {0}, ind = 0;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            if (finish[i] == 0) {
                int flag = 0;
                for (int j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]){ flag = 1; break; }
                }
                if (flag == 0) {
                    safe[ind++] = i;
                    for (int y = 0; y < m; y++) avail[y] += alloc[i][y];
                    finish[i] = 1;
                }
            }
        }
    }

    printf("Safe Sequence: ");
    for (int i = 0; i < n - 1; i++) printf("P%d -> ", safe[i]);
    printf("P%d\n", safe[n - 1]);
    return 0;
}
```

Output - 7

```
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

8.1) First In First Out (FIFO) algorithm

```
#include <stdio.h>

int main() {
    int ref[] = {1, 3, 0, 3, 5, 6}, frames[3], faults = 0, n = 6, f = 3, k =
0;
    for(int i = 0; i < f; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int hit = 0;
        for (int j = 0; j < f; j++) {
            if (frames[j] == ref[i]) { hit = 1; break; }
        }
        if (!hit) {
            frames[k] = ref[i];
            k = (k + 1) % f;
            faults++;
        }
        printf("Ref %d -> Frames: [%d, %d, %d]\n", ref[i], frames[0],
frames[1], frames[2]);
    }
    printf("Total Page Faults: %d\n", faults);
    return 0;
}
```

Output - 8.1

```
Ref 1 -> Frames: [1, -1, -1]
Ref 3 -> Frames: [1, 3, -1]
Ref 0 -> Frames: [1, 3, 0]
Ref 3 -> Frames: [1, 3, 0]
Ref 5 -> Frames: [5, 3, 0]
Ref 6 -> Frames: [5, 6, 0]
Total Page Faults: 5
```

8.2) Least Recently Used (LRU) algorithm

```
#include <stdio.h>

int main() {
    int ref[] = {1, 2, 3, 2, 1, 5, 2, 1, 6}, frames[3], time[3], n = 9, f = 3;
    int faults = 0, counter = 0;
    for(int i = 0; i < f; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int hit = 0, pos = -1, min = counter;
        for (int j = 0; j < f; j++) {
            if (frames[j] == ref[i]) { hit = 1; time[j] = ++counter; break; }
        }
        if (!hit) {
            for (int j = 0; j < f; j++) {
                if (frames[j] == -1) { pos = j; break; }
                if (time[j] < min) { min = time[j]; pos = j; }
            }
            frames[pos] = ref[i];
            time[pos] = ++counter;
            faults++;
        }
        printf("Ref %d -> Frames: [%d, %d, %d]\n", ref[i], frames[0],
frames[1], frames[2]);
    }
    printf("Total Page Faults: %d\n", faults);
    return 0;
}
```

Output - 8.2

```
Ref 1 -> Frames: [1, -1, -1]
Ref 2 -> Frames: [1, 2, -1]
Ref 3 -> Frames: [1, 2, 3]
Ref 2 -> Frames: [1, 2, 3]
Ref 1 -> Frames: [1, 2, 3]
Ref 5 -> Frames: [5, 2, 3]
Ref 2 -> Frames: [5, 2, 3]
Ref 1 -> Frames: [5, 2, 1]
Ref 6 -> Frames: [6, 2, 1]
Total Page Faults: 6
```

9.1 / 11.1) First Come First Serve (FCFS) algorithm (Disk Scheduling)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int req[] = {82, 170, 43, 140, 24, 16, 190}, head = 50, thm = 0;
    int n = 7;

    printf("FCFS Sequence: %d", head);
    for (int i = 0; i < n; i++) {
        thm += abs(head - req[i]);
        head = req[i];
        printf(" -> %d", head);
    }
    printf("\nTotal Head Movement: %d\n", thm);
    return 0;
}
```


Output - 9.1 / 11.1

```
FCFS Sequence: 50 -> 82 -> 170 -> 43 -> 140 -> 24 -> 16 -> 190  
Total Head Movement: 642
```

9.2 / 11.2) Shortest Seek Time First (SSTF) algorithm

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int req[] = {82, 170, 43, 140, 24, 16, 190}, head = 50, n = 7;
    int thm = 0, visited[7] = {0};

    printf("SSTF Sequence: %d", head);
    for (int i = 0; i < n; i++) {
        int min = 1000, min_idx = -1;
        for (int j = 0; j < n; j++) {
            if (!visited[j] && abs(head - req[j]) < min) {
                min = abs(head - req[j]);
                min_idx = j;
            }
        }
        visited[min_idx] = 1;
        thm += min;
        head = req[min_idx];
        printf(" -> %d", head);
    }
    printf("\nTotal Head Movement: %d\n", thm);
    return 0;
}
```

Output - 9.2 / 11.2

```
SSTF Sequence: 50 -> 43 -> 24 -> 16 -> 82 -> 140 -> 170 -> 190  
Total Head Movement: 208
```

11.3) SCAN algorithm

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int req[] = {82, 170, 43, 140, 24, 16, 190}, head = 50, disk_size = 200, n
= 7;
    int thm = 0, temp;

    for(int i=0; i<n-1; i++)
        for(int j=i+1; j<n; j++)
            if(req[i] > req[j]) { temp=req[i]; req[i]=req[j]; req[j]=temp; }

    printf("SCAN Sequence: %d", head);
    int pos = 0;
    while (req[pos] < head) pos++;

    for (int i = pos; i < n; i++) {
        thm += abs(head - req[i]); head = req[i];
        printf(" -> %d", head);
    }
    thm += abs(head - (disk_size - 1)); head = disk_size - 1;
    printf(" -> %d", head);

    for (int i = pos - 1; i >= 0; i--) {
        thm += abs(head - req[i]); head = req[i];
        printf(" -> %d", head);
    }
    printf("\nTotal Head Movement: %d\n", thm);
    return 0;
}
```

Output - 11.3

```
SCAN Sequence: 50 -> 82 -> 140 -> 170 -> 190 -> 199 -> 43 -> 24 ->  
16  
Total Head Movement: 332
```